Do notebook ao projeto – consulta rápida

1 · Ambiente e dependências

Anexo A2 · §2 a §6
macOS e Linux: `python3`

01 EXECUTAR CÓDIGO

A2 · §2–3

`python --version`	Confirmar a instalação e a versão.
`py -3.13 ficheiro.py`	Windows: escolher a versão com o *launcher*.
`python ficheiro.py Ana`	Executar um script; `Ana` chega em `sys.argv[1]`.
`python -m pytest`	Executar um módulo pelo nome, com este Python.
`python -c "print(2 ** 10)"`	Executar uma linha de código.
`import sys` `sys.version  sys.executable`	No notebook: que Python está a correr, e onde.

02 AMBIENTES VIRTUAIS

A2 · §4

	Windows (PowerShell)	macOS e Linux
Criar	`python -m venv .venv`	`python3 -m venv .venv`
Ativar	`.venv\Scripts\Activate.ps1`	`source .venv/bin/activate`
Desativar	`deactivate`	`deactivate`

`(.venv) $`	Com o ambiente ativo, o terminal mostra o nome; python e pip passam a ser os do ambiente.
`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`	Windows: uma vez, se o PowerShell recusar o `Activate.ps1`.
`Python: Select Interpreter`	VS Code: escolher o Python da `.venv` (e o kernel dos notebooks).

03 ALTERNATIVAS

A2 · §2

`conda create -n proj python` `conda activate proj`	Anaconda / Miniconda: ambientes conda.
`uv venv  uv pip install pandas` `uv run script.py`	uv: rápido; gere versões de Python, ambientes e dependências.

04 DEPENDÊNCIAS COM PIP

A2 · §5

`python -m pip install pandas`	Instalar no Python do ambiente ativo.

Abreviado: `pip` = `python -m pip`

`pip install "pandas>=2.2,<4"`	Uma versão dentro de um intervalo.
`pip install --upgrade pandas`	Atualizar.
`pip list  pip show pandas`	Instaladas · detalhes de uma.
`pip freeze > requirements.txt`	Guardar as versões exatas.
`pip install -r requirements.txt`	Instalar tudo o que está na lista.
`pip install -e .`	Instalar o projeto em modo editável.
`%pip install pandas`	No notebook (no Python do kernel).

Versões no `requirements.txt`

`pandas==3.0.1`	Exatamente esta (reprodutível).
`numpy>=2.0  matplotlib>=3.8,<4`	Mínimo · intervalo.
`scikit-learn~=1.6`	Compatível: `>=1.6` e `<2.0`.

05 ESTRUTURA DE UM PROJETO

A2 · §6

`projeto/`
`├── pyproject.toml  README.md`
`├── requirements.txt  .gitignore`
`├── src/pacote/`
`│  ├── __init__.py`
`│  ├── __main__.py`
`│  └── modulo.py`
`└── tests/test_modulo.py`
	src layout: o código importa-se como pacote.
	`__main__.py`: `python -m pacote`.
	`__init__.py`: o que o pacote exporta.

`[project]`
`name = "pacote"`
`dependencies = ["pandas>=2.2"]`
	`pyproject.toml`: projeto, dependências e configuração das ferramentas.

Uma `.venv` por projeto. Usa sempre `python -m pip`: garante que instalas no mesmo Python que vai correr o código.

Do notebook ao projeto – consulta rápida

06 TESTES COM PYTEST

A2 · §7

```
# tests/test_estatistica.py
from pacote import media

def test_media():
    assert media([10, 12, 14]) == 12
```

Ficheiros e funções começam por `test_`; usa-se o `assert` normal.

`assert m == pytest.approx(0.15)`	Comparar decimais com tolerância.
`with pytest.raises(ValueError): media([])`	O teste passa se o bloco lançar o erro.
`@pytest.mark.parametrize("n, esp", [ (9.5, "Aprovado"), (9.4, "Reprovado")])`	O mesmo teste com vários valores.

Correr os testes

`python -m pytest`	Todos os testes da pasta.
`-v  -q  -x  -k media`	Detalhado · resumido · parar no 1.º erro · só os que têm «media».

07 QUALIDADE DO CÓDIGO: RUFF

A2 · §8

`ruff check .`	*Linter*: erros prováveis e más práticas, sem executar.
`ruff check . --fix`	Corrigir o que for seguro.
`ruff format .  ruff format . --diff`	Formatar (estilo do `black`) · só mostrar.
`python -m ruff check .`	Se o comando `ruff` não for reconhecido.
`F401  E711  I001  UP031`	Import não usado · == None · ordem dos imports · sintaxe antiga.

09 VOLTAR AO NOTEBOOK

A2 · §10

`python -m pip install -e .`	O notebook passa a importar o pacote do projeto.
`%load_ext autoreload` `%autoreload 2`	Recarregar os módulos alterados sem reiniciar.

08 GIT

A2 · §9

`git config --global user.name "Ana"` `git config --global user.email a@x.pt`	Uma vez por computador.
`git init -b main`	Transformar a pasta num repositório.
`git status  git diff`	O que mudou · que linhas mudaram.
`git add .  git add ficheiro.py`	Preparar as alterações (*staging*).
`git commit -m "Acrescenta a média"`	Guardar um ponto no histórico.
`git log --oneline`	Histórico, um commit por linha.
`git switch -c nova-funcao`	Criar um ramo e mudar para ele.

GitHub

`git remote add origin URL` `git push -u origin main`	Ligar ao repositório remoto · primeiro envio.
`git push  git pull  git clone URL`	Enviar · receber · copiar um repositório.

! ARMADILHAS FREQUENTES

pip no Python errado	Usa `python -m pip` e confirma `sys.executable`.
.venv no Git	Põe-na no `.gitignore`; recria-se com o `requirements.txt`.
Segredos no Git	Ficam no histórico para sempre. Usa um `.env` ignorado.
Notebooks com saídas	Limpa as saídas antes do commit (ou `nbstripout`).